

LAPORAN TUGAS MATA KULIAH BUSINESS INTELLIGENCE



Dosen Pembimbing :

Akhmad Irsyad S.T.,M.Kom

Disusun Oleh :

Muhammad Rizky Febrianto	2409116045
Dinathan Fahrezi	2409116050
Nur Ihsan	2409116051
Ahmad Dani	2409116074

**PROGRAM STUDI SISTEM INFORMASI
FAKULTAS TEKNIK
UNIVERSITAS MULAWARMAN
2026/2027**

KATA PENGANTAR

Puji syukur penulis panjatkan kehadirat Tuhan Yang Maha Esa karena atas rahmat dan karunia-Nya laporan tugas mata kuliah *Business Intelligence* ini dapat diselesaikan dengan baik dan tepat waktu.

Laporan ini disusun sebagai bentuk pemenuhan tugas mata kuliah *Business Intelligence* yang membahas mengenai proses ETL (*Extract, Transform, Load*) serta perancangan *data warehouse* menggunakan model *Star Schema* pada dataset *Superstore*. Dalam penyusunan laporan ini, dilakukan berbagai tahapan mulai dari pengumpulan data, proses *Extract, Transform* data, hingga *load* data ke dalam database untuk mendukung proses analisis data dan *Business Intelligence*.

Penulis menyampaikan ucapan terima kasih yang sebesar-besarnya kepada Bapak Akhmad Irsyad S.T.,M.Kom selaku dosen pengampu mata kuliah *Business Intelligence* yang telah memberikan ilmu, arahan, serta bimbingan selama proses perkuliahan dan penyusunan laporan ini. Ucapan terima kasih juga penulis sampaikan kepada seluruh pihak yang telah memberikan dukungan dan bantuan, baik secara langsung maupun tidak langsung, sehingga laporan ini dapat diselesaikan dengan baik.

Penulis menyadari bahwa laporan ini masih memiliki kekurangan, baik dari segi isi maupun penyusunannya. Oleh karena itu, penulis sangat mengharapkan kritik dan saran yang membangun guna penyempurnaan laporan di masa mendatang.

Akhir kata, penulis berharap laporan ini dapat memberikan manfaat serta menambah wawasan dalam bidang *Business Intelligence*, khususnya terkait penerapan proses ETL dan *data warehouse* menggunakan model *Star Schema*.

Samarinda, Mei 2026

Penulis

DAFTAR ISI

KATA PENGANTAR.....	ii
DAFTAR ISI	iii
DAFTAR GAMBAR.....	iv
BAB I PENDAHULUAN.....	1
1.1 Latar Belakang.....	1
1.2 Rumusan Masalah.....	2
1.3 Tujuan	2
BAB II DATA DAN METOLOGI	3
2.1 Deskripsi Dataset	3
2.2 Tahapan penelitian.....	4
2.3 Perancangan <i>Warehouse</i>	5
2.3.1 <i>Fact Table</i>	5
2.3.2 <i>Dimension Table</i>	6
2.3.3 Relasi Antar Tabel	7
2.3.4 Alasan Penggunaan <i>Star Schema</i>	7
2.3 Proses ETL (<i>Extract, Transform, Load</i>)	8
2.4.1 <i>Extract</i>	8
2.4.2 <i>Transform</i>	8
2.4.3 Load	12
BAB III HASIL DAN PEMBAHASAN	17
3.1 Hasil.....	17
3.1.1 Hasil <i>Extract</i>	17
3.1.2 Hasil <i>Transform</i>	18
3.1.3 Hasil Load.....	20
3.1.4 Hasil Anilisis.....	20
3.2 Pembahasan	22
BAB IV PENUTUP.....	24
4.1. Kesimpulan	24
4.2. Saran	25

DAFTAR GAMBAR

Gambar 2. 1 <i>Star Schema</i>	7
Gambar 2. 2 Import Library.....	8
Gambar 2. 3 Pengecekan Struktur Data.....	8
Gambar 2. 4 Standarisasi Nama Kolom.....	9
Gambar 2. 5 Konversi Tipe Data	10
Gambar 2. 6 Pengecekan <i>Missing values</i>	10
Gambar 2. 7 Menjaga Konsistensi Penulisan Data.....	11
Gambar 2. 8 <i>feature engineering</i> Kolom <i>shipping_days</i>	11
Gambar 2. 9 Menambahkan Label Bulan dan Tahun	11
Gambar 2. 10 Pembuatan Atribut <i>Revenue</i>	12
Gambar 2. 11 Pembuatan Atribut Profit	12
Gambar 2. 12 Pengecekan dan Penyesuaian Data	13
Gambar 2. 13 Membuat <i>Dimension Table</i> dan <i>Fact Table</i>	14
Gambar 2. 14 Membuat <i>Dimension Table</i> dan <i>Fact Table</i>	15
Gambar 2. 15 Membuat <i>Dimension Table</i> dan <i>Fact Table</i>	15
Gambar 2. 16 Ekspor Kedalam File SQL	16
Gambar 3. 1 Hasil <i>Extract</i>	17
Gambar 3. 2 Hasil <i>Transform</i>	18
Gambar 3. 3 Hasil Load.....	20
Gambar 3. 4 Analisis Total Sales per Year.....	20
Gambar 3. 5 Analisis Top 10 Produk Berdasarkan Profit	21
Gambar 3. 6 Analisis Sales per Region	21

BAB I

PENDAHULUAN

1.1 Latar Belakang

Data penjualan merupakan salah satu aset penting bagi perusahaan karena dapat digunakan untuk mengetahui kondisi bisnis, perilaku pelanggan, serta performa produk yang dijual. Setiap transaksi yang terjadi menghasilkan data dalam jumlah besar yang terus bertambah seiring berjalannya waktu. Jika data tersebut tidak dikelola dengan baik, maka perusahaan akan mengalami kesulitan dalam melakukan analisis dan memperoleh informasi yang dibutuhkan untuk mendukung pengambilan keputusan. Oleh karena itu, diperlukan suatu sistem yang mampu mengelola dan menyusun data secara terstruktur agar lebih mudah dianalisis.

Business Intelligence merupakan salah satu pendekatan yang digunakan untuk mengolah dan menganalisis data sehingga menghasilkan informasi yang lebih akurat, mudah dipahami, dan mendukung kebutuhan bisnis. Dalam penerapannya, *Business Intelligence* memanfaatkan *data warehouse* sebagai media penyimpanan data terintegrasi yang dirancang untuk kebutuhan analisis. Salah satu model *data warehouse* yang umum digunakan adalah *Star Schema* karena memiliki struktur sederhana yang terdiri dari *Fact Table* dan *Dimension Table* sehingga mempermudah proses analisis multidimensi. Selain itu, proses ETL (*Extract, Transform, Load*) juga berperan penting dalam pengolahan data, mulai dari pengambilan data, pembersihan dan Transformasi data, hingga pemuatan data ke dalam *database*.

Pada laporan ini digunakan dataset *Superstore* yang berisi data transaksi penjualan retail, seperti data pelanggan, produk, lokasi, penjualan, dan profit. Dataset tersebut digunakan untuk menerapkan proses ETL dan perancangan *data warehouse* menggunakan model *Star Schema*. Melalui proses tersebut, data yang sebelumnya masih berupa data mentah dapat diolah menjadi struktur data yang lebih terorganisir dan siap dianalisis. Hasil analisis diharapkan mampu memberikan insight terkait performa penjualan, *profit*, kategori produk, serta wilayah penjualan yang dapat mendukung proses pengambilan keputusan bisnis.

1.2 Rumusan Masalah

Berdasarkan latar belakang yang telah dijelaskan, maka rumusan masalah dalam laporan ini adalah sebagai berikut:

1. Bagaimana proses ETL (*Extract, Transform, Load*) dilakukan pada dataset *Superstore*?
2. Bagaimana perancangan *data warehouse* menggunakan model *Star Schema* pada dataset *Superstore*?
3. Bagaimana proses pembentukan *Fact Table* dan *Dimension Table* dalam *data warehouse*?
4. Bagaimana proses Transformasi data agar sesuai dengan struktur *data warehouse* yang dirancang?
5. Bagaimana hasil analisis data penjualan berdasarkan *data warehouse* yang telah dibuat?

1.3 Tujuan

Berdasarkan rumusan masalah yang telah disusun, maka tujuan dari laporan ini adalah untuk memahami penerapan proses ETL dan perancangan *data warehouse* dalam mendukung analisis data penjualan. Adapun tujuan dari laporan ini adalah sebagai berikut:

1. Untuk memahami proses ETL (*Extract, Transform, Load*) pada dataset *Superstore*.
2. Untuk merancang *data warehouse* menggunakan model *Star Schema*.
3. Untuk membentuk *Fact Table* dan *Dimension Table* sesuai kebutuhan analisis data.
4. Untuk melakukan proses Transformasi data agar sesuai dengan struktur *data warehouse* yang dirancang.
5. Untuk menganalisis data penjualan berdasarkan *data warehouse* yang telah dibuat.

BAB II

DATA DAN METOLOGI

2.1 Deskripsi Dataset

Dataset yang digunakan dalam laporan ini adalah dataset *Superstore* yang berisi data transaksi penjualan pada sebuah perusahaan retail. Dataset ini digunakan untuk kebutuhan analisis Business Intelligence dan penerapan proses ETL (*Extract, Transform, Load*) serta perancangan *data warehouse* menggunakan model *Star Schema*. Data yang digunakan terdiri dari berbagai informasi terkait transaksi penjualan, pelanggan, produk, lokasi, dan keuntungan penjualan.

Dataset *Superstore* memiliki beberapa atribut penting yang digunakan dalam proses analisis, yaitu sebagai berikut:

- Order ID , Merupakan kode unik yang digunakan untuk mengidentifikasi setiap transaksi penjualan.
- Order Date , Menunjukkan tanggal dilakukannya transaksi pemesanan produk oleh pelanggan.
- Ship Date , Menunjukkan tanggal pengiriman produk kepada pelanggan.
- Customer Name , Berisi nama pelanggan yang melakukan transaksi pembelian.
- Segment, Menunjukkan kategori pelanggan, seperti *Consumer*, *Corporate*, atau *Home Office*.
- Product Name , Berisi nama produk yang dijual dalam transaksi.
- Category, Menunjukkan kategori utama produk, seperti *Furniture*, *Office Supplies*, dan *Technology*.
- Sub-Category, Menjelaskan jenis produk yang lebih spesifik berdasarkan kategori utama.
- Sales, Menunjukkan total nilai penjualan yang dihasilkan dari suatu transaksi.
- Quantity, Menunjukkan jumlah produk yang dibeli dalam satu transaksi.
- Profit, Menunjukkan keuntungan atau kerugian yang diperoleh dari transaksi penjualan.
- Region, Menunjukkan wilayah penjualan produk.
- State, Menunjukkan nama negara bagian atau provinsi tempat transaksi terjadi.
- City, Menunjukkan kota tempat pelanggan melakukan transaksi pembelian.

Sebelum digunakan dalam proses analisis, dataset terlebih dahulu melalui tahap data cleaning untuk memastikan data dalam kondisi baik dan siap diolah. Proses tersebut meliputi pengecekan *missing value*, penghapusan data duplikat, serta penyesuaian format data agar sesuai dengan kebutuhan *data warehouse*. Setelah proses pembersihan selesai, data kemudian digunakan dalam proses ETL dan dimodelkan ke dalam bentuk *Star Schema* yang terdiri dari *Fact Table* dan *Dimension Table* untuk mempermudah proses analisis data.

2.2 Tahapan penelitian

Penelitian ini dilakukan melalui beberapa tahapan yang sistematis untuk memastikan data dapat diolah dan dianalisis dengan baik menggunakan pendekatan *Business Intelligence*. Tahapan penelitian dilakukan mulai dari pengumpulan data hingga proses analisis data menggunakan *data warehouse*. Adapun tahapan penelitian yang dilakukan adalah sebagai berikut:

1. Pengumpulan Data

Pada tahap ini, data dikumpulkan dalam bentuk dataset *Superstore* yang berisi informasi transaksi penjualan retail. Dataset diperoleh dari sumber publik dan digunakan sebagai bahan utama dalam proses ETL dan analisis data.

2. Proses ETL (*Extract, Transform, Load*)

Tahap ETL dilakukan untuk mengolah data dari sumber data hingga siap digunakan dalam *data warehouse*. Proses ETL terdiri dari:

- *Extract*, yaitu proses mengambil data dari file CSV.
- *Transform*, yaitu proses membersihkan, menyesuaikan, dan memisahkan data ke dalam *Fact Table* dan *Dimension Table*.
- *Load*, yaitu proses memasukkan data yang telah diTransformasikan ke dalam database SQLite.

3. Perancangan *Data warehouse*

Pada tahap ini dilakukan perancangan *data warehouse* menggunakan model *Star Schema*. Struktur *data warehouse* terdiri dari satu *Fact Table* sebagai pusat data transaksi dan beberapa *Dimension Table* yang menyimpan informasi deskriptif. Perancangan ini bertujuan untuk mempermudah proses analisis data dan meningkatkan efisiensi query.

4. Analisis Data

Tahap terakhir adalah melakukan analisis terhadap data yang telah dimasukkan ke dalam *data warehouse*. Analisis dilakukan untuk memperoleh insight terkait penjualan, profit, performa produk, serta distribusi penjualan berdasarkan wilayah dan pelanggan.

2.3 Perancangan *Warehouse*

Perancangan *data warehouse* pada penelitian ini menggunakan model *Star Schema*. Model ini dipilih karena memiliki struktur yang sederhana dan mudah dipahami sehingga mempermudah proses analisis data dalam penerapan *Business Intelligence*. *Star Schema* terdiri dari satu *Fact Table* sebagai pusat data transaksi dan beberapa *Dimension Table* yang berisi informasi deskriptif pendukung.

Dalam perancangan ini, *Fact Table* digunakan untuk menyimpan data utama transaksi penjualan, sedangkan *Dimension Table* digunakan untuk menyimpan informasi pelanggan, produk, lokasi, dan waktu transaksi. Struktur tersebut memungkinkan proses analisis data dilakukan dengan lebih cepat dan efisien.

2.3.1 *Fact Table*

Fact Table merupakan tabel utama yang menyimpan data transaksi penjualan. Tabel ini berisi data numerik yang akan digunakan dalam proses analisis, seperti total penjualan, jumlah produk, dan *profit*. *Fact Table* juga memiliki *Foreign key* yang terhubung ke *Dimension Table*. Nama *Fact Table* yang digunakan adalah *fact_sales*. Atribut yang terdapat pada *Fact Table* adalah sebagai berikut:

- *order_id*

Merupakan identitas unik untuk setiap transaksi penjualan.

- *customer_id*

Foreign key yang menghubungkan data transaksi dengan tabel pelanggan.

- *product_id*

Foreign key yang menghubungkan data transaksi dengan tabel produk.

- *location_id*

Foreign key yang menghubungkan data transaksi dengan tabel lokasi.

- *date_id*

Foreign key yang menghubungkan data transaksi dengan tabel waktu.

- sales
Menunjukkan total nilai penjualan pada transaksi.
- quantity
Menunjukkan jumlah produk yang dibeli dalam transaksi.
- profit
Menunjukkan keuntungan yang diperoleh dari transaksi penjualan.

2.3.2 Dimension Table

Dimension Table digunakan untuk menyimpan informasi deskriptif yang mendukung *Fact Table*. Tabel dimensi membantu proses analisis data berdasarkan kategori tertentu.

1. dim_customer

Tabel ini digunakan untuk menyimpan informasi pelanggan. Atribut:

- customer_id, Primary key pelanggan
- customer_name, Nama pelanggan
- segment, Kategori pelanggan

2. dim_product

Tabel ini digunakan untuk menyimpan informasi produk.

Atribut:

- product_id, Primary key produk
- product_name, Nama produk
- category, Kategori produk
- sub_category, Sub kategori produk

3. dim_location

Tabel ini digunakan untuk menyimpan informasi lokasi transaksi. Atribut:

- location_id, Primary key lokasi
- city, Kota transaksi
- state, Negara bagian/provinsi
- region, Wilayah penjualan

4. dim_date

Tabel ini digunakan untuk menyimpan informasi waktu transaksi. Atribut:

- date_id, Primary key tanggal

- order_date, Tanggal transaksi
- month, Bulan transaksi
- year, Tahun transaksi

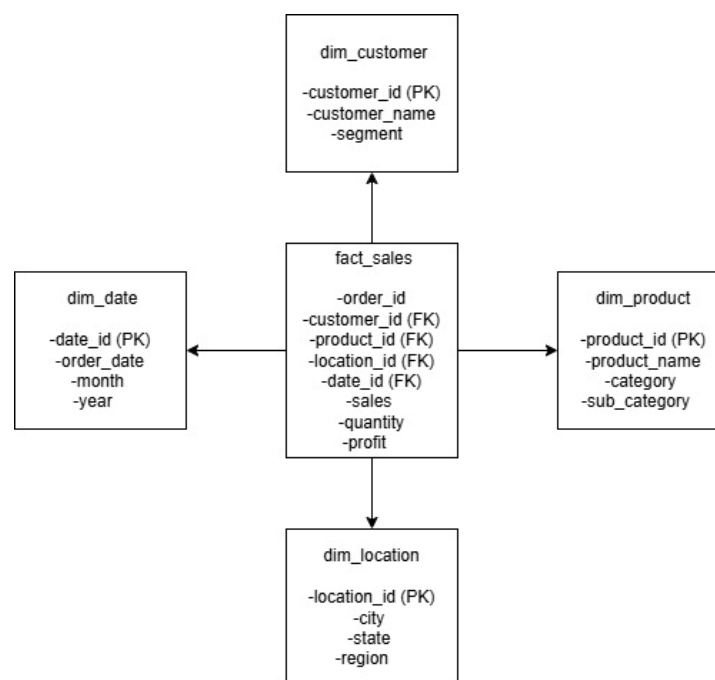
2.3.3 Relasi Antar Tabel

Pada model *Star Schema*, *Fact Table* berada di pusat dan terhubung langsung dengan seluruh *Dimension Table* melalui *foreign key*. Relasi ini memungkinkan proses analisis multidimensi dilakukan dengan lebih mudah, seperti analisis penjualan berdasarkan produk, pelanggan, lokasi, maupun waktu transaksi.

2.3.4 Alasan Penggunaan *Star Schema*

Model *Star Schema* dipilih dalam penelitian ini karena memiliki beberapa kelebihan, yaitu:

- Struktur tabel sederhana dan mudah dipahami
- Mempermudah proses query dan analisis data
- Memiliki performa yang baik dalam proses pengolahan data
- Mendukung analisis multidimensi pada *Business Intelligence*
- Mempermudah proses visualisasi dan pelaporan data



Gambar 2. 1 *Star Schema*

2.3 Proses ETL (*Extract, Transform, Load*)

Proses ETL (*Extract, Transform, Load*) merupakan tahapan penting dalam pembangunan *data warehouse*. Proses ini bertujuan untuk mengambil data dari sumber data, mengolah data agar sesuai dengan struktur *data warehouse*, kemudian memuat data tersebut ke dalam *database* sehingga siap digunakan untuk proses analisis *Business Intelligence*.

2.4.1 *Extract*

Pada tahap *Extract*, data diambil dari sumber data berupa file CSV dataset *Superstore* menggunakan bahasa pemrograman Python dengan bantuan library Pandas. Proses *Extract* dilakukan untuk membaca dan menampilkan dataset sebelum dilakukan pengolahan lebih lanjut.

Tahapan *Extract* meliputi proses import library, membaca file CSV menggunakan fungsi `read_csv()`, serta menampilkan beberapa data awal untuk memastikan dataset berhasil dimuat ke dalam dataframe.

Selain itu, dilakukan juga pengecekan struktur data untuk mengetahui jumlah kolom, tipe data, dan kondisi dataset sebelum memasuki tahap Transformasi.

```
import pandas as pd
import numpy as np
from datetime import datetime
import os

print('Library berhasil dimuat.')

Library berhasil dimuat.
```

Gambar 2. 2 Import Library

Jumlah baris : 9994

Jumlah kolom : 29

Kolom yang tersedia:

{ 'row_id', 'order_id', 'order_date', 'ship_date', 'ship_mode', 'customer_id', 'customer_name', 'segment', 'country', 'city', 'state', 'postal_code', 'region', 'product_id', 'category', 'sub_category', 'product_name', 'sales', 'quantity', 'discount' }

Contoh 3 baris pertama:

row_id	order_id	order_date	ship_date	ship_mode	customer_id	customer_name	segment	country	city	...	discount	profit	shipping_days	profit_ratio	revenue	order_year	order_month	order_quarter	month_label	is_profitable	
0	1	CA-2016-152156	2016-11-08	2016-11-11	Second Class	CG-12520	Chloe Gule	Consumer	United States	Henderson	...	0.0	41.9136	3	0.16	261.96	2016	11	4	Nov 2016	1
1	2	CA-2016-152156	2016-11-08	2016-11-11	Second Class	CG-12520	Chloe Gule	Consumer	United States	Henderson	...	0.0	219.5620	3	0.30	731.94	2016	11	4	Nov 2016	1
2	3	CA-2016-138658	2016-06-12	2016-06-16	Second Class	DV-13045	Darren Van Huff	Corporate	United States	Los Angeles	...	0.0	6.8714	4	0.47	14.62	2016	6	2	Jun 2016	1

3 rows x 29 columns

Gambar 2. 3 Pengecekan Struktur Data

2.4.2 *Transform*

Tahap *Transform* dilakukan untuk mengolah data agar sesuai dengan struktur *data warehouse* yang telah dirancang menggunakan model *Star Schema*. Pada tahap ini dilakukan proses pembersihan data, pemilihan atribut yang relevan, serta pembentukan *Dimension Table* dan *Fact Table*.

Proses Transformasi diawali dengan data cleaning, seperti pengecekan *missing value*, penghapusan data duplikat, dan penyesuaian format data. Setelah data dinyatakan bersih, data kemudian dipisahkan ke dalam beberapa tabel dimensi dan *Fact Table* sesuai kebutuhan analisis.

Dimension Table yang dibuat terdiri dari:

- dim_customer
- dim_product
- dim_location
- dim_date

Sedangkan *Fact Table* yang digunakan adalah:

- fact_sales

Tahap *Transform* bertujuan untuk menghasilkan struktur data yang lebih terorganisir sehingga mempermudah proses query dan analisis data.

```
Nama kolom setelah standarisasi:
['res_id', 'order_id', 'order_date', 'ship_date', 'ship_mode', 'customer_id', 'customer_name', 'segment', 'country', 'city', 'state', 'postal_code', 'region', 'product_id', 'category', 'sub_category', 'product_name', 'sales', 'quantity', 'discount']
```

Gambar 2. 4 Standarisasi Nama Kolom

Pada tahapan *Transform* dilakukan proses pembersihan dan penyesuaian data agar struktur data menjadi lebih rapi dan konsisten sebelum digunakan dalam *data warehouse*. Salah satu proses yang dilakukan adalah penyesuaian format nama kolom agar memiliki penulisan yang seragam dan lebih mudah digunakan pada proses pengolahan data selanjutnya. Tahap ini bertujuan untuk mempermudah proses pembuatan tabel, mengurangi kemungkinan terjadinya kesalahan, serta membuat data lebih terstruktur sehingga mendukung proses analisis

```

df['order_date'] = pd.to_datetime(df['order_date'], format='%m/%d/%Y')
df['ship_date'] = pd.to_datetime(df['ship_date'], format='%m/%d/%Y')

# Konversi postal_code ke string (kode pos bukan angka matematis)
df['postal_code'] = df['postal_code'].astype(str).str.zfill(5)

# Verifikasi tipe data setelah konversi
print(df[['order_date', 'ship_date', 'postal_code']].dtypes)
print(df[['order_date', 'ship_date']].head(3))

```

```

order_date    datetime64[ns]
ship_date     datetime64[ns]
postal_code    object
dtype: object
   order_date  ship_date
0 2016-11-08 2016-11-11
1 2016-11-08 2016-11-11
2 2016-06-12 2016-06-16

```

Gambar 2. 5 Konversi Tipe Data

Penyesuaian tipe data dilakukan agar setiap atribut memiliki format yang sesuai dengan kebutuhan analisis. Proses ini meliputi pengubahan kolom tanggal ke dalam format datetime sehingga data waktu dapat digunakan dengan lebih baik pada proses analisis, serta penyesuaian beberapa atribut lain agar format datanya tetap konsisten.

```

missing = df.isnull().sum()
print('Missing values per kolom:')
print(missing[missing > 0] if missing.any() else 'Tidak ada missing value.')

# — CELL 10: Cek & hapus duplikasi —
dup_count = df.duplicated().sum()
print(f'Jumlah baris duplikat: {dup_count}')

if dup_count > 0:
    df = df.drop_duplicates()
    print(f'Duplikat dihapus. Baris tersisa: {len(df)}')
else:
    print('Tidak ada duplikat.')

```

```

Missing values per kolom:
Tidak ada missing value.
Jumlah baris duplikat: 0
Tidak ada duplikat.

```

Gambar 2. 6 Pengecekan *Missing values*

Pengecekan nilai kosong (*missing value*) serta penghapusan data yang duplikat agar tidak terjadi pengulangan data pada proses analisis. Tahap ini penting dilakukan untuk menjaga keakuratan dan konsistensi data sehingga hasil analisis yang dihasilkan menjadi lebih valid dan dapat digunakan dengan baik dalam proses *Business Intelligence*.

whitespace pada kolom teks berhasil dibersihkan.
kolom yang dibersihkan: ['order_id', 'ship_mode', 'customer_id', 'customer_name', 'segment', 'country', 'city', 'state', 'postal_code', 'region', 'product_id', 'category', 'sub_category', 'product_name']

Gambar 2. 7 Menjaga Konsistensi Penulisan Data

Menghapus spasi yang tidak diperlukan pada beberapa kolom kategori, seperti nama pelanggan, produk, dan wilayah. Tahap ini bertujuan untuk menjaga konsistensi data serta menghindari terjadinya perbedaan penulisan yang dapat mempengaruhi hasil pengolahan dan analisis data.

```
Feature Engineering
Shipping Days (Lama Pengiriman)
1.1 df['shipping_days'] = (df['ship_date'] - df['order_date']).dt.days
    print('Distribusi shipping days:')
    print(df['shipping_days'].describe())

Distribusi shipping_days:
count    9994.000000
mean      3.958175
std       1.747657
min       0.000000
25%       3.000000
50%       4.000000
75%       5.000000
max       7.000000
Name: shipping_days, dtype: float64
```

Gambar 2. 8 *feature engineering* Kolom shipping_days

feature engineering digunakan untuk menambahkan atribut baru berupa lama pengiriman (shipping_days). Atribut ini diperoleh dari selisih antara tanggal pengiriman dan tanggal pemesanan sehingga dapat digunakan untuk mengetahui durasi pengiriman pada setiap transaksi. Penambahan atribut ini bertujuan untuk mendukung analisis terkait efisiensi proses pengiriman dan distribusi produk.

```
Kolom Waktu (Year, Month, Quarter)
1.1 df['order_year'] = df['order_date'].dt.year
    df['order_month'] = df['order_date'].dt.month
    df['order_quarter'] = df['order_date'].dt.quarter

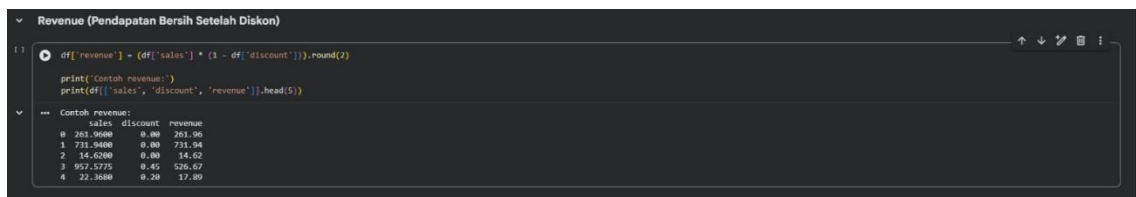
    # Tambahkan label bulan untuk kemudahan visualisasi
    df['month_label'] = df['order_date'].dt.strftime('%b %Y') # Contoh: 'Nov 2016'

    print('Komponen waktu berhasil di ekstrak:')
    print(df[['order_date', 'order_year', 'order_month', 'order_quarter', 'month_label']].head(5))

Komponen waktu berhasil di ekstrak:
   order_date order_year order_month order_quarter month_label
0 2016-11-08      2016         11           4      Nov 2016
1 2016-11-08      2016         11           4      Nov 2016
2 2016-06-12      2016          6           2      Jun 2016
3 2015-10-11      2015         10           4      Oct 2015
4 2015-10-11      2015         10           4      Oct 2015
```

Gambar 2. 9 Menambahkan Label Bulan dan Tahun

Feature engineering juga dilakukan dengan membuat atribut waktu seperti tahun, bulan, kuartal, dan nama bulan berdasarkan data tanggal transaksi. Penambahan atribut waktu ini bertujuan untuk mempermudah proses analisis data berdasarkan periode tertentu sehingga tren penjualan dapat dianalisis dengan lebih baik.



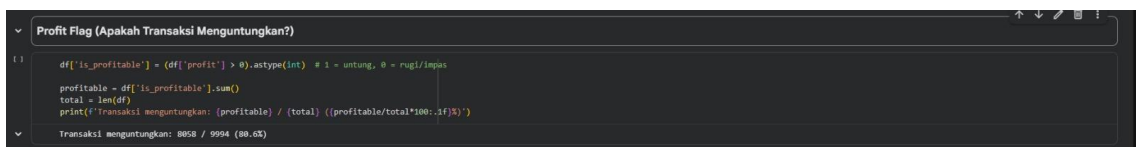
```
df['revenue'] = (df['sales'] * (1 - df['discount'])).round(2)

print('Contoh revenue:')
print(df[['sales', 'discount', 'revenue']].head(5))
```

	sales	discount	revenue
0	261.9680	0.00	261.96
1	731.9480	0.00	731.94
2	14.6280	0.00	14.62
3	957.5775	0.45	526.67
4	22.3680	0.20	17.89

Gambar 2. 10 Pembuatan Atribut *Revenue*

Selain itu, dilakukan juga pembuatan atribut *revenue* yang digunakan untuk mengetahui pendapatan bersih setelah memperhitungkan diskon pada transaksi penjualan. Atribut ini membantu menghasilkan informasi keuangan yang lebih akurat sehingga dapat mendukung proses analisis performa penjualan dan pendapatan perusahaan.



```
df['is_profitable'] = (df['profit'] > 0).astype(int) # 1 = untung, 0 = rugi/lepas

profitable = df['is_profitable'].sum()
total = len(df)
print(f'Transaksi menguntungkan: {profitable} / {total} ({profitable/total*100:.1f}%)')

Transaksi menguntungkan: 8858 / 9994 (88.63)
```

Gambar 2. 11 Pembuatan Atribut Profit

Pada tahapan *Transform* juga dilakukan pembuatan atribut *profit_ratio* yang digunakan untuk mengetahui rasio keuntungan dari setiap transaksi penjualan. Nilai tersebut diperoleh dari perbandingan antara profit dan sales sehingga dapat memberikan gambaran mengenai tingkat keuntungan yang dihasilkan. Penambahan atribut ini membantu proses analisis profitabilitas agar hasil analisis menjadi lebih mudah dipahami.

2.4.3 Load

Pada tahap load, data yang telah diTransformasikan dimasukkan ke dalam database SQLite menggunakan Python. Proses load dilakukan dengan membuat koneksi database kemudian memuat seluruh tabel dimensi dan *Fact Table* ke dalam database menggunakan fungsi `to_sql()`.

Tahap ini bertujuan agar *data warehouse* dapat digunakan untuk proses analisis data dan query *Business Intelligence* secara lebih efisien. Tabel yang dimuat ke dalam database meliputi:

- `dim_customer`
- `dim_product`
- `dim_location`
- `dim_date`

- fact_sales

Setelah proses load selesai, dilakukan pengecekan untuk memastikan seluruh tabel berhasil dimasukkan ke dalam database dengan baik.

```
# Buat canonical mapping (ambil data pertama per key)
prod_canonical = df[['product_id', 'product_name', 'category', 'sub_category']] \
    .groupby('product_id', as_index=False).first()

loc_canonical = df[['postal_code', 'city', 'state', 'region', 'country']] \
    .groupby('postal_code', as_index=False).first()

# Update df dengan nilai canonical
df = df.merge(prod_canonical[['product_id', 'product_name']] \
    .rename(columns={'product_name': 'pn_fix'}), on='product_id', how='left')
df['product_name'] = df['pn_fix']
df.drop(columns=['pn_fix'], inplace=True)

df = df.merge(loc_canonical[['postal_code', 'city']] \
    .rename(columns={'city': 'city_fix'}), on='postal_code', how='left')
df['city'] = df['city_fix']
df.drop(columns=['city_fix'], inplace=True)

print('Verifikasi (max harus = 1):')
print('product_id:', df.groupby('product_id')['product_name'].nunique().max())
print('postal_code:', df.groupby('postal_code')['city'].nunique().max())

Verifikasi (max harus = 1):
product_id: 1
postal_code: 1
```

Gambar 2. 12 Pengecekan dan Penyesuaian Data

Pada tahapan load dilakukan proses pengecekan dan penyesuaian data untuk memastikan tidak terjadi duplikasi pada key yang digunakan dalam tabel dimensi. Proses ini bertujuan untuk menjaga konsistensi hubungan antar tabel sehingga data yang dimasukkan ke dalam *data warehouse* tetap terstruktur dengan baik. Dengan memastikan setiap key memiliki nilai yang unik, proses penggabungan data antar tabel dapat dilakukan dengan lebih akurat dan mengurangi kemungkinan terjadinya inkonsistensi data.

```

# — CELL 19: Buat dimension tables —
dim_customers = df[['customer_id', 'customer_name', 'segment']] \
    .drop_duplicates().reset_index(drop=True)
dim_customers.insert(0, 'customer_pk', range(1, len(dim_customers)+1))

dim_products = df[['product_id', 'product_name', 'category', 'sub_category']] \
    .drop_duplicates().reset_index(drop=True)
dim_products.insert(0, 'product_pk', range(1, len(dim_products)+1))

dim_location = df[['postal_code', 'city', 'state', 'region', 'country']] \
    .drop_duplicates().reset_index(drop=True)
dim_location.insert(0, 'location_pk', range(1, len(dim_location)+1))

dates = df[['order_date']].drop_duplicates().copy()
dates['date_pk'] = dates['order_date'].dt.strftime('%Y%m%d').astype(int)
dates['year'] = dates['order_date'].dt.year
dates['month'] = dates['order_date'].dt.month
dates['quarter'] = dates['order_date'].dt.quarter
dates['month_name'] = dates['order_date'].dt.strftime('%B')
dates['day_of_week'] = dates['order_date'].dt.day_name()
dim_date = dates.rename(columns={'order_date': 'full_date'}).reset_index(drop=True)

# — CELL 20: Buat fact table dengan FK —
df_fact = df.merge(dim_customers[['customer_id', 'customer_pk']], on='customer_id', how='left')
df_fact = df_fact.merge(dim_products[['product_id', 'product_pk']], on='product_id', how='left')
df_fact = df_fact.merge(dim_location[['postal_code', 'location_pk']], on='postal_code', how='left')
df_fact['date_pk'] = df_fact['order_date'].dt.strftime('%Y%m%d').astype(int)

fact_orders = df_fact[[
    'row_id', 'order_id', 'date_pk', 'ship_date', 'ship_mode',
    'customer_pk', 'product_pk', 'location_pk',
    'sales', 'quantity', 'discount', 'profit',
    'shipping_days', 'profit_ratio', 'revenue', 'is_profitable'
]].copy()

print('fact_orders shape:', fact_orders.shape)

fact_orders shape: (9994, 16)

```

Gambar 2. 13 Membuat *Dimension Table* dan *Fact Table*

Pada tahap ini juga dilakukan proses pembentukan tabel dimensi dan tabel fakta sebagai bagian dari perancangan *data warehouse* menggunakan model *Star Schema*. Data dipisahkan ke dalam beberapa tabel dimensi seperti pelanggan, produk, lokasi, dan waktu, kemudian dihubungkan ke tabel fakta yang berisi data transaksi penjualan. Struktur ini bertujuan untuk mempermudah proses analisis multidimensi sehingga data dapat diolah dan dianalisis dengan lebih cepat dan efisien.

```

def escape_sql(value):
    if isinstance(value, str):
        value = value.replace('\\', '\\\\')
        value = value.replace("'", "\\'")
    return value

def df_to_insert_sql(df, table_name, batch_size=500):
    sql_lines = []
    cols = ', '.join([f'`{c}`' for c in df.columns])
    for start in range(0, len(df), batch_size):
        batch = df.iloc[start:start+batch_size]
        values_list = []
        for _, row in batch.iterrows():
            vals = []
            for v in row:
                if pd.isna(v) if not isinstance(v, str) else False: vals.append('NULL')
                elif isinstance(v, str): vals.append(f'`{escape_sql(v)}`')
                elif hasattr(v, 'strftime'): vals.append(f'"{v.strftime("%Y-%m-%d")}"')
                elif isinstance(v, (int, np.integer)): vals.append(str(int(v)))
                else: vals.append(str(round(float(v), 4)))
            values_list.append(f'({", ".join(vals)})')
        sql_lines.append(f'INSERT INTO `{table_name}` ({cols}) VALUES\n' +
            ',\n'.join(values_list) + ';')
    return '\n\n'.join(sql_lines)

```

Gambar 2. 14 Membuat *Dimension Table* dan *Fact Table*

Selanjutnya dilakukan proses persiapan penyimpanan data ke dalam format SQL agar struktur *data warehouse* dapat digunakan pada sistem basis data relasional. Proses ini meliputi penyesuaian struktur tabel, tipe data, serta relasi antar tabel agar sesuai dengan hasil Transformasi yang telah dilakukan sebelumnya. Tahap ini bertujuan untuk memastikan seluruh tabel dimensi dan tabel fakta dapat disimpan dan digunakan dengan baik pada *database*.

```

Membuat File SQL

11 def escape_sql(value):
12     if isinstance(value, str):
13         value = value.replace('\\', '\\\\')
14         value = value.replace("'", "\\'")
15     return value
16
17 def df_to_insert_sql(df, table_name, batch_size=500):
18     sql_lines = []
19     cols = ', '.join([f'`{c}`' for c in df.columns])
20     for start in range(0, len(df), batch_size):
21         batch = df.iloc[start:start+batch_size]
22         values_list = []
23         for _, row in batch.iterrows():
24             vals = []
25             for v in row:
26                 if pd.isna(v) if not isinstance(v, str) else False: vals.append('NULL')
27                 elif isinstance(v, str): vals.append(f'`{escape_sql(v)}`')
28                 elif hasattr(v, 'strftime'): vals.append(f'"{v.strftime("%Y-%m-%d")}"')
29                 elif isinstance(v, (int, np.integer)): vals.append(str(int(v)))
30                 else: vals.append(str(round(float(v), 4)))
31             values_list.append(f'({", ".join(vals)})')
32         sql_lines.append(f'INSERT INTO `{table_name}` ({cols}) VALUES\n' +
33             ',\n'.join(values_list) + ';')
34     return '\n\n'.join(sql_lines)
35
36 11 DDL = ""
37 12 SET NAMES utf8mb4;
38 13 SET FOREIGN_KEY_CHECKS = 0;
39 14 SET SQL_MODE = "NO_AUTO_VALUE_ON_ZERO";
40 15
41 16 DROP DATABASE IF EXISTS superstore_db;
42 17 CREATE DATABASE superstore_db
43 18 CHARACTER SET utf8mb4
44 19 COLLATE utf8mb4_unicode_ci;
45 20 USE superstore_db;
46
47 21 CREATE TABLE `dim_customers` (
48 22   `customer_pk` INT NOT NULL AUTO_INCREMENT,
49 23   `customer_id` VARCHAR(20) NOT NULL,
50 24   `customer_name` VARCHAR(100) NOT NULL,
51 25   `region` VARCHAR(50) NOT NULL,
52 26   PRIMARY KEY (`customer_pk`))
53 27 ENGINE=InnoDB;

```

Gambar 2. 15 Membuat *Dimension Table* dan *Fact Table*

```
11 dia_date_sql = dia_data.copy()
12 dia_date_sql['full_date'] = dia_date_sql['full_date'].dt.strftime('%Y-%m-%d')
13 fact_sql = fact_orders.copy()
14 fact_sql['ship_date'] = fact_sql['ship_date'].dt.strftime('%Y-%m-%d')
15
16 # (DDL tersedia di file fixed_etl_superstore.py)
17 full_sql = '\n\n'.join([DDL,
18     '-- dia_customers\n' + df.to_insert_sql(dia_customers, 'dia_customers'),
19     '-- dia_products\n' + df.to_insert_sql(dia_products, 'dia_products'),
20     '-- dia_location\n' + df.to_insert_sql(dia_location, 'dia_location'),
21     '-- dia_date\n' + df.to_insert_sql(dia_date_sql, 'dia_date'),
22     '-- fact_orders\n' + df.to_insert_sql(fact_sql, 'fact_orders'),
23 ])
24
25 with open('superstore_dw_fixed.sql', 'w', encoding='utf-8') as f:
26     f.write(full_sql)
27 files.download('superstore_dw_fixed.sql')
28 print('File SQL berhasil dibuat dan didownload!')
```

Gambar 2. 16 Ekspor Kedalam File SQL

Tahap akhir dilakukan dengan mengekspor seluruh struktur tabel dan data ke dalam file SQL yang siap digunakan untuk implementasi *data warehouse*. Pada tahap ini dilakukan penyesuaian format data agar sesuai dengan standar database, kemudian seluruh tabel dan isi data digabungkan ke dalam satu file SQL.

BAB III

HASIL DAN PEMBAHASAN

3.1 Hasil

Proses ETL (*Extract, Transform, Load*) yang dilakukan pada dataset *Superstore* berhasil menghasilkan data yang lebih terstruktur dan siap digunakan dalam pembangunan *data warehouse* menggunakan model *Star Schema*. Tahapan ETL dilakukan secara bertahap mulai dari pengambilan data, pembersihan dan Transformasi data, hingga proses penyimpanan data ke dalam format SQL.

3.1.1 Hasil *Extract*

```
=== INFO TYPE DATA ===
Row ID          int64
Order ID        object
Order Date      object
Ship Date       object
Ship Mode       object
Customer ID     object
Customer Name   object
Segment        object
Country        object
City           object
State          object
Postal Code     int64
Region         object
Product ID     object
Category       object
Sub-Category   object
Product Name    object
Sales          float64
Quantity       int64
Discount       float64
Profit         float64
dtype: object

=== MISSING VALUES ===
Row ID          0
Order ID        0
Order Date      0
Ship Date       0
Ship Mode       0
Customer ID     0
Customer Name   0
Segment        0
Country        0
City           0
State          0
Postal Code     0
Region         0
Product ID     0
Category       0
Sub-Category   0
Product Name    0
Sales          0
Quantity       0
Discount       0
Profit         0
dtype: int64

=== STATISTIK DESKRIPTIF ===
              Sales      Quantity      Discount      Profit
count  9994.000000  9994.000000  9994.000000  9994.000000
mean    229.858001    3.789574    0.156203    28.656896
std     623.245101    2.225110    0.206452    234.260108
min       0.444000     1.000000    0.000000  -6599.978000
25%      17.280000     2.000000    0.000000    1.728750
50%      54.490000     3.000000    0.200000    8.666500
75%     209.940000     5.000000    0.200000    29.364000
max    22638.480000    14.000000    0.800000   8399.976000

=== NILAI UNIK KOLOM KATEGORIS ===
Ship Mode: ['Second Class' 'Standard Class' 'First Class' 'Same Day']
Segment:   ['Consumer' 'Corporate' 'Home Office']
```

Gambar 3. 1 Hasil *Extract*

Pada tahap *Extract*, dataset *Superstore* berhasil diunggah dan dibaca ke dalam Google Colab menggunakan Python. Data yang berhasil dimuat terdiri dari informasi transaksi penjualan, pelanggan, produk, lokasi, dan profit penjualan. Berdasarkan hasil pengecekan dataset, diketahui bahwa dataset memiliki berbagai tipe data seperti integer, object, dan float yang digunakan sesuai dengan kebutuhan masing-masing atribut. Selain itu, hasil pengecekan *missing value* menunjukkan bahwa seluruh atribut tidak memiliki nilai kosong sehingga dataset dapat langsung digunakan pada tahap Transformasi. Tahap *Extract* juga menampilkan statistik deskriptif dari beberapa atribut numerik seperti sales, quantity, discount, dan profit. Informasi tersebut digunakan untuk mengetahui gambaran umum data sebelum dilakukan proses pengolahan lebih lanjut.

3.1.2 Hasil *Transform*

```

=== RINGKASAN DATASET SETELAH TRANSFORM ===
Jumlah baris : 9994
Jumlah kolom : 29
Kolom baru : shipping_days, profit_ratio, revenue,
            order_year, order_month, order_quarter,
            month_label, is_profitable

Tipe data final:
row_id          int64
order_id        object
order_date      datetime64[ns]
ship_date       datetime64[ns]
ship_mode       object
customer_id     object
customer_name   object
segment        object
country        object
city           object
state          object
postal_code     object
region         object
product_id     object
category       object
sub_category   object
product_name   object
sales          float64
quantity       int64
discount       float64
profit         float64
shipping_days  int64
profit_ratio   float64
revenue        float64
order_year     int32
order_month    int32
order_quarter  int32
month_label    object
is_profitable  int64
dtype: object

Dataset bersih disimpan ke superstore_clean.csv

```

Gambar 3. 2 Hasil *Transform*

Pada tahap *Transform* dilakukan proses pembersihan dan penyesuaian data agar data menjadi lebih rapi dan sesuai dengan struktur *data warehouse*. Proses *Transform* diawali dengan standarisasi nama kolom dan pembersihan whitespace pada data teks

untuk menjaga konsistensi format data. Selain itu, dilakukan juga penyesuaian tipe data pada beberapa atribut, khususnya atribut tanggal agar dapat digunakan untuk analisis berbasis waktu. Setelah proses pembersihan selesai, dilakukan feature engineering dengan menambahkan beberapa atribut baru seperti:

- shipping_days
- profit_ratio
- revenue
- order_year
- order_month
- order_quarter
- month_label
- is_profitable

Penambahan atribut tersebut bertujuan untuk memperkaya informasi analisis sehingga proses *Business Intelligence* dapat dilakukan dengan lebih efektif.

Hasil *Transform* menunjukkan bahwa dataset akhir memiliki:

- Jumlah baris: 9994
- Jumlah kolom: 29

Selain itu, dataset hasil *Transform* berhasil disimpan ke dalam file *Superstore_clean.csv* yang nantinya digunakan dalam proses pembentukan *data warehouse*. Setelah proses *Transform* selesai, data kemudian dipisahkan ke dalam beberapa tabel dimensi dan tabel fakta sesuai dengan model *Star Schema*. Tabel dimensi yang berhasil dibuat meliputi:

- dim_customer
- dim_product
- dim_location
- dim_date

Sedangkan tabel fakta yang digunakan adalah:

- fact_orders

3.1.3 Hasil Load

```
dim_date_sql = dim_date.copy()
dim_date_sql['full_date'] = dim_date_sql['full_date'].dt.strftime('%Y-%m-%d')
fact_sql = fact_orders.copy()
fact_sql['ship_date'] = fact_sql['ship_date'].dt.strftime('%Y-%m-%d')

# (DDL tersedia di file fixed_etl_superstore.py)
full_sql = '\n\n'.join([DDL,
    '-- dim_customers\n' + df.to_insert_sql(dim_customers, 'dim_customers'),
    '-- dim_products\n' + df.to_insert_sql(dim_products, 'dim_products'),
    '-- dim_location\n' + df.to_insert_sql(dim_location, 'dim_location'),
    '-- dim_date\n' + df.to_insert_sql(dim_date_sql, 'dim_date'),
    '-- fact_orders\n' + df.to_insert_sql(fact_sql, 'fact_orders'),
])

with open('superstore_dw_fixed.sql', 'w', encoding='utf-8') as f:
    f.write(full_sql)
files.download('superstore_dw_fixed.sql')
print('File SQL berhasil dibuat dan didownload!')
```

Gambar 3. 3 Hasil Load

Pada tahap load dilakukan proses penyimpanan seluruh tabel hasil Transformasi ke dalam format SQL. Struktur tabel dan data yang telah dibentuk sebelumnya berhasil dikonversi menjadi file SQL sehingga siap digunakan pada sistem database relasional. Hasil akhir dari proses load berupa file *Superstore_dw_fixed.sql* yang berisi tabel dimensi, tabel fakta, serta data transaksi yang telah disesuaikan dengan struktur *data warehouse*. File tersebut dapat digunakan untuk kebutuhan analisis data dan pengembangan dashboard Business Intelligence.

3.1.4 Hasil Analisis

```
mysql> SELECT d.year, SUM(f.sales) AS total_sales FROM fact_orders f JOIN dim_date d ON f.date_pk = d.date_pk GROUP BY d.year ORDER BY d.year;
```

year	total_sales
2014	484247.56
2015	470532.46
2016	609205.86
2017	733215.19

4 rows in set (0.20 sec)

Gambar 3. 4 Analisis Total Sales per Year

Berdasarkan hasil query *Total Sales per Year*, diketahui bahwa total penjualan mengalami peningkatan dari tahun ke tahun, meskipun sempat mengalami sedikit penurunan pada tahun 2015. Pada tahun 2014 total penjualan tercatat sebesar 484.247,56, kemudian menurun menjadi 470.532,46 pada tahun 2015. Setelah itu, penjualan kembali meningkat pada tahun 2016 menjadi 609.205,86 dan mencapai nilai tertinggi pada tahun 2017 sebesar 733.215,19.

Hasil ini menunjukkan bahwa performa penjualan perusahaan cenderung mengalami pertumbuhan positif dalam jangka panjang. Peningkatan yang signifikan pada tahun 2016 dan 2017 mengindikasikan bahwa strategi penjualan atau permintaan pasar terhadap produk perusahaan mengalami perkembangan yang cukup baik.


```
mysql> SELECT p.product_name, SUM(f.profit) AS total_profit FROM fact_orders f JOIN dim_products p ON f.product_pk = p.product_pk GROUP BY p.product_name ORDER BY total_profit DESC LIMIT 10;
```

product_name	total_profit
Canon imageCLASS 2200 Advanced Copier	25199.9280
Fellowes PB500 Electric Punch Plastic Comb Binding Machine with Manual Bind	7753.0390
Hewlett Packard LaserJet 3310 Copier	6983.8836
Canon PC1060 Personal Laser Copier	4570.9347
Logitech G19 Programmable Gaming Keyboard	4425.3432
HP Designjet T520 Inkjet Large Format Printer - 24" Color	4094.9766
Ativa V4110MDD Micro-Cut Shredder	3772.9461
3D Systems Cube Printer, 2nd Generation, Magenta	3717.9714
Ibico EPK-21 Electric Binding System	3345.2823
Zebra ZM400 Thermal Label Printer	3343.5360

10 rows in set (0.16 sec)

Gambar 3. 5 Analisis Top 10 Produk Berdasarkan Profit

Berdasarkan hasil query *Top 10 Produk Berdasarkan Profit*, produk dengan keuntungan tertinggi adalah *Canon imageCLASS 2200 Advanced Copier* dengan total profit mencapai 25.199,9280. Nilai tersebut jauh lebih tinggi dibandingkan produk lainnya, sehingga dapat disimpulkan bahwa produk tersebut memberikan kontribusi keuntungan terbesar bagi perusahaan.

Selain itu, beberapa produk lain yang juga memberikan profit tinggi berasal dari kategori perangkat kantor dan teknologi seperti printer, copier, binding machine, serta gaming keyboard. Hal ini menunjukkan bahwa produk teknologi dan perlengkapan kantor memiliki kontribusi profit yang cukup besar terhadap pendapatan perusahaan. Dari hasil tersebut, perusahaan dapat memprioritaskan produk-produk dengan profit tinggi untuk strategi pemasaran, pengelolaan stok, maupun pengembangan penjualan agar keuntungan perusahaan dapat terus meningkat.

```
mysql> SELECT l.region, SUM(f.sales) AS total_sales FROM fact_orders f JOIN dim_location l ON f.location_pk = l.location_pk GROUP BY l.region;
```

region	total_sales
South	391721.90
West	725457.93
Central	501239.88
East	678781.36

4 rows in set (0.20 sec)

Gambar 3. 6 Analisis Sales per Region

Berdasarkan hasil query *Sales per Region*, wilayah dengan total penjualan tertinggi adalah wilayah West dengan total sales sebesar 725.457,93. Posisi berikutnya diikuti oleh wilayah East dengan total penjualan sebesar 678.781,36. Sementara itu, wilayah Central memiliki total penjualan sebesar 501.239,88 dan wilayah South menjadi wilayah dengan total penjualan terendah yaitu sebesar 391.721,90.

Hasil ini menunjukkan bahwa performa penjualan perusahaan paling kuat berada pada wilayah West dan East. Sebaliknya, wilayah South memiliki performa penjualan

yang relatif lebih rendah dibandingkan wilayah lainnya. Informasi ini dapat dimanfaatkan perusahaan untuk mengevaluasi strategi pemasaran dan distribusi produk pada setiap wilayah agar penjualan dapat lebih merata.

3.2 Pembahasan

Berdasarkan proses ETL yang telah dilakukan, dataset *Superstore* berhasil diolah menjadi data yang lebih terstruktur dan siap digunakan dalam pembangunan *data warehouse* menggunakan model *Star Schema*. Proses *Extract* berhasil mengambil seluruh data transaksi penjualan dari file CSV ke dalam lingkungan kerja Google Colab tanpa kendala. Hasil pengecekan dataset menunjukkan bahwa data tidak memiliki *missing value* sehingga kualitas data awal dapat dikatakan cukup baik untuk digunakan pada proses pengolahan selanjutnya. Selain itu, struktur data yang terdiri dari atribut pelanggan, produk, lokasi, penjualan, dan profit memberikan informasi yang cukup lengkap untuk kebutuhan analisis *Business Intelligence*.

Pada tahap *Transform* dilakukan berbagai proses pembersihan dan penyesuaian data agar data menjadi lebih konsisten dan mudah diolah. Proses standarisasi nama kolom dan pembersihan *whitespace* membantu menjaga keseragaman format data sehingga mengurangi kemungkinan terjadinya kesalahan saat proses pengolahan maupun query data. Penyesuaian tipe data, khususnya pada atribut tanggal, juga mempermudah proses analisis berbasis waktu karena data dapat diolah berdasarkan bulan, kuartal, maupun tahun transaksi.

Selain proses pembersihan data, tahap *Transform* juga melakukan *feature engineering* untuk menghasilkan atribut baru yang mendukung proses analisis data. Atribut *shipping_days* memungkinkan analisis terkait lama pengiriman produk sehingga dapat digunakan untuk mengevaluasi efisiensi distribusi barang. Atribut *profit_ratio* membantu mengetahui tingkat keuntungan dari setiap transaksi penjualan, sedangkan atribut *revenue* memberikan informasi pendapatan yang lebih akurat setelah memperhitungkan diskon transaksi. Penambahan atribut waktu seperti *order_year*, *order_month*, dan *order_quarter* juga membantu proses analisis tren penjualan berdasarkan periode tertentu.

Hasil *Transform* menunjukkan bahwa dataset akhir memiliki 9994 baris data dengan total 29 kolom setelah penambahan beberapa atribut baru. Hal ini menunjukkan bahwa

proses Transformasi berhasil memperkaya informasi data tanpa mengurangi jumlah data transaksi yang tersedia. Dataset hasil *Transform* kemudian disimpan ke dalam file *Superstore_clean.csv* untuk digunakan pada tahap pembentukan *data warehouse*.

Pada tahap pembentukan *data warehouse*, data berhasil dipisahkan ke dalam tabel dimensi dan tabel fakta menggunakan model *Star Schema*. Tabel dimensi digunakan untuk menyimpan informasi deskriptif seperti pelanggan, produk, lokasi, dan waktu transaksi, sedangkan tabel fakta digunakan untuk menyimpan data utama transaksi penjualan seperti sales, quantity, profit, dan revenue. Struktur *Star Schema* yang digunakan membuat hubungan antar data menjadi lebih sederhana dan mempermudah proses analisis multidimensi.

Berdasarkan hasil statistik deskriptif pada dataset, diketahui bahwa nilai penjualan (sales) memiliki rata-rata sekitar 229,85 dengan nilai maksimum mencapai lebih dari 22 ribu. Hal ini menunjukkan adanya perbedaan yang cukup besar antara transaksi kecil dan transaksi dengan nilai penjualan tinggi. Selain itu, nilai profit juga menunjukkan variasi yang cukup besar, di mana terdapat transaksi yang menghasilkan keuntungan tinggi namun terdapat juga transaksi yang mengalami kerugian. Kondisi ini menunjukkan bahwa tidak semua penjualan memberikan keuntungan yang optimal bagi perusahaan.

Analisis lain juga menunjukkan bahwa jumlah produk yang dibeli pada setiap transaksi relatif kecil dengan rata-rata sekitar 3 unit per transaksi. Hal ini mengindikasikan bahwa sebagian besar transaksi dilakukan dalam jumlah pembelian yang tidak terlalu besar. Selain itu, adanya atribut *is_profitable* memungkinkan proses identifikasi transaksi yang menguntungkan maupun merugikan dilakukan dengan lebih cepat sehingga perusahaan dapat melakukan evaluasi terhadap strategi penjualan dan pemberian diskon.

Tahap load menjadi tahap akhir dalam proses ETL, di mana seluruh tabel dimensi dan tabel fakta berhasil dikonversi ke dalam format SQL dan disimpan dalam file *Superstore_dw_fixed.sql*. Hasil ini menunjukkan bahwa *data warehouse* telah berhasil dibangun dan siap digunakan pada sistem database relasional untuk kebutuhan analisis data, visualisasi dashboard, maupun pengambilan keputusan bisnis. Dengan adanya *data warehouse* yang terstruktur, proses analisis data dapat dilakukan dengan lebih cepat, konsisten, dan efisien dibandingkan menggunakan data mentah secara langsung.

BAB IV

PENUTUP

4.1. Kesimpulan

Dari hasil proses ETL (*Extract, Transform, Load*) yang telah dilakukan pada dataset *Superstore*, dapat disimpulkan bahwa proses pengolahan data berhasil dilakukan dengan baik mulai dari pengambilan data, pembersihan data, Transformasi data, hingga pembentukan *data warehouse* menggunakan model *Star Schema*. Tahap *Extract* berhasil mengambil data transaksi penjualan dari file CSV ke dalam lingkungan kerja Google Colab tanpa ditemukan *missing value* sehingga data dapat langsung digunakan pada proses selanjutnya.

Pada tahap *Transform*, dilakukan berbagai proses pembersihan dan penyesuaian data seperti standarisasi nama kolom, penyesuaian tipe data, penghapusan data duplikat, serta penambahan beberapa atribut baru melalui *feature engineering*. Proses tersebut berhasil membuat data menjadi lebih terstruktur, konsisten, dan siap digunakan untuk kebutuhan analisis *Business Intelligence*. Selain itu, penambahan atribut seperti *shipping_days*, *profit_ratio*, *revenue*, dan atribut waktu membantu memperkaya informasi yang dapat digunakan dalam proses analisis data.

Perancangan *data warehouse* menggunakan model *Star Schema* juga berhasil dilakukan dengan membentuk beberapa tabel dimensi dan tabel fakta yang saling terhubung. Struktur ini mempermudah proses pengolahan dan analisis data berdasarkan berbagai aspek seperti pelanggan, produk, lokasi, dan waktu transaksi. Hasil akhir dari proses ETL berupa file SQL menunjukkan bahwa *data warehouse* berhasil dibangun dan siap digunakan untuk kebutuhan analisis data, visualisasi *dashboard*, serta mendukung proses pengambilan keputusan bisnis secara lebih efektif dan efisien.

Berdasarkan rumusan masalah yang telah disusun sebelumnya, maka dapat disimpulkan juga sebagai berikut:

1. Proses ETL pada dataset *Superstore* berhasil dilakukan melalui tahapan *Extract, Transform*, dan *load* sehingga data mentah dapat diolah menjadi data yang lebih terstruktur dan siap digunakan dalam analisis *Business Intelligence*.

2. Perancangan *data warehouse* menggunakan model *Star Schema* berhasil diterapkan dengan membentuk satu tabel fakta dan beberapa tabel dimensi yang saling terhubung untuk mendukung proses analisis multidimensi.
3. Proses pembentukan *Fact Table* dan *Dimension Table* berhasil dilakukan dengan memisahkan data transaksi dan data deskriptif ke dalam struktur tabel yang lebih terorganisir sehingga mempermudah pengelolaan dan analisis data.
4. Proses Transformasi data berhasil meningkatkan kualitas data melalui *data cleaning*, standarisasi nama kolom, penyesuaian tipe data, penghapusan data duplikat, serta penambahan atribut baru seperti *shipping_days*, *profit_ratio*, *revenue*, dan atribut waktu.
5. Hasil analisis data penjualan menunjukkan adanya variasi nilai penjualan dan profit pada setiap transaksi sehingga data dapat digunakan untuk menganalisis performa bisnis berdasarkan pelanggan, produk, wilayah, maupun periode waktu tertentu.

4.2. Saran

Berdasarkan hasil analisis data dan proses ETL yang telah dilakukan pada dataset *Superstore*, terdapat beberapa saran yang dapat digunakan untuk meningkatkan performa bisnis maupun pengembangan sistem analisis data, yaitu sebagai berikut:

1. Perusahaan disarankan untuk melakukan evaluasi terhadap transaksi yang memiliki profit rendah atau mengalami kerugian agar dapat diketahui faktor penyebabnya, seperti pemberian diskon yang terlalu besar atau tingginya biaya operasional.
2. Perusahaan dapat meningkatkan pengawasan terhadap proses pengiriman produk, terutama pada transaksi yang memiliki waktu pengiriman cukup lama, sehingga efisiensi distribusi dan kepuasan pelanggan dapat ditingkatkan.
3. Analisis penjualan berdasarkan kategori produk dan wilayah penjualan perlu dilakukan secara berkala untuk mengetahui produk serta wilayah dengan performa terbaik sehingga strategi pemasaran dapat dilakukan dengan lebih tepat.
4. Perusahaan dapat memanfaatkan *data warehouse* yang telah dibangun untuk memantau performa penjualan pada setiap wilayah, kategori produk, dan pelanggan sehingga strategi bisnis dapat disusun dengan lebih tepat berdasarkan data yang tersedia.